
UWB RTLS — Full Improvement Plan

Revision 2 | June 2026

Hardware	4× Makerfabs ESP32 UWB Pro with Display (DW1000)
Configuration	3 anchors + 1 tag (extendable)
Goal	XYZ + RPY indoor positioning, no cameras, microcontroller-level
Current state	3–10 cm XY accuracy, 2D only, no NLOS detection, no IMU
Target	≤ 3 cm XY, ≤ 5 cm Z, $\leq 5^\circ$ RPY

Contents

1	Context and System Summary	2
2	Existing Library Comparison	2
2.1	Capability Matrix	2
2.2	Detailed Library Analysis	2
2.2.1	thotro/arduino-dw1000 (canonical, abandoned May 2020)	2
2.2.2	Makerfabs mf_DW1000	3
2.2.3	jremington/UWB-Indoor-Localization_Arduino	3
2.2.4	F-Army/arduino-dw1000-ng (archived November 2023)	3
2.2.5	NLOS Detection — Verified Novelty	3
2.3	Techniques Borrowed from Existing Libraries	4
3	Implementation Status	4
4	Phase 1 — Foundation Fixes	4
4.1	1.0 Switch to 64MHz PRF Accuracy Mode Easy	4
4.2	1.1 Better Calibration Easy	4
4.3	1.2 Adaptive Polling + Age-Gated Anchor Eviction Easy	5
4.4	1.3 Faster Sweep Rate Medium	5
4.5	1.4 Multipath Bias Correction ✓ Done	5
4.6	1.5 Anchor Self-Survey (Auto-Localization) Medium	5
5	Phase 2 — Ranging Quality	6
5.1	2.1 FP_POWER NLOS Detection Medium ★ Novel	6
5.2	2.2 Weighted Multilateration Medium	6
5.3	2.3 NLOS Range Bias Correction Easy	6
5.4	2.4 Per-Anchor Diagnostics Panel Easy	6
5.5	2.5 GPS-Style δ -Solve (4+ Anchors) Medium	7
6	Phase 3 — State Estimation	7
6.1	3.1 Constant-Acceleration EKF (CV $\rightarrow$ CA Model) Medium	7
6.2	3.2 δ as EKF State (Works with 3 Anchors) Med-Hard	7
6.3	3.3 Moving Horizon Estimator (MHE) Hard	7
6.4	3.4 Hybrid EKF + MHE Medium	7
7	Phase 4 — 3D + Orientation (XYZ + RPY)	8
7.1	4.1 4th Anchor + 3D Configuration Easy	8
7.2	4.2 BNO085 IMU — Firmware Implementation Medium	8
7.3	4.3 RPY Output from Quaternion Easy	8
7.4	4.4 Loose Coupling — IMU-Aided EKF Hard	8
7.5	4.5 Tight Coupling — Raw Ranges + IMU in One Filter Very Hard	9
8	Phase 5 — System Hardening	9
8.1	5.1 Deep Sleep on Anchors (from dw1000-ng) Medium	9
8.2	5.2 DW1000 Driver Error Handling Medium	9
8.3	5.3 Temperature Compensation Medium	9
8.4	5.4 Online Self-Calibration Hard	9
9	Phase 6 — Scale and Future	9
9.1	6.1 Multi-Tag TDMA Superframe Hard	9
9.2	6.2 Differential Timestamp Encoding (pizzo00 technique) Hard	9

9.3 6.3 On-Device 2D Solve (jremington technique)	Medium	9
10 Full Item Index		10
11 Expected Accuracy at Each Phase		10
12 Recommended Implementation Sequence		11

1 Context and System Summary

The system uses Ultra-Wideband (UWB) radio time-of-flight ranging to locate a mobile tag relative to fixed anchors. The DW1000 chip achieves 15.65 ps timestamp resolution (4.7 mm per tick), giving a theoretical 1–2 cm floor in LOS conditions. Current performance of 3–10 cm is dominated by calibration errors, uncorrected multipath bias, and the absence of NLOS detection. This plan describes how to close that gap in six phases.

All position computation runs on the host PC (MATLAB). The tag firmware streams raw distances; no algorithm changes are required on the microcontroller to add more anchors or change the estimator.

2 Existing Library Comparison

2.1 Capability Matrix

Table 1: Open-source DW1000 library capability comparison (verified against current source, June 2026)

Capability	thotro	Makerfabs	jremington	pizzo00	dw1000-ng	UwbRtls (or)
Anchor ceiling	4	4	4 [†]	6	No limit [‡]	No limit
RF collisions	Yes	Yes	Yes	Yes	No	No
Update rate	0.6 Hz	0.6 Hz	0.6 Hz	1 Hz	2 Hz	3+ Hz
Multi-tag	No	No	No	Partial	No	Designed-
NLOS detection	No	No	No	No	No	Planned (2)
IMU fusion	No	No	No	No	No	Planned (4)
3D + RPY	No	No	Fragile	No	No	Planned (4.1)
Anchor self-survey	No	No	No	No	No	Planned (1)
Actively maintained	No (2020)	Partial	Yes	Yes	No (arch. 2023)	Yes

[†] jremington raises `MAX_DEVICES` to 7 but the `RANGE` frame encoding still overflows at 5 anchors — see §2.2.

[‡] dw1000-ng uses individual unicast TWR (no shared frame), so no frame-size ceiling. Archived November 2023, single-tag only.

2.2 Detailed Library Analysis

2.2.1 thotro/arduino-dw1000 (canonical, abandoned May 2020)

- **4-anchor hard ceiling.** The `RANGE` frame packs 17 bytes per device (2-byte address + three 5-byte absolute timestamps) into a single 90-byte buffer (`LEN_DATA = 90`). With the 9-byte MAC header: $\lfloor (90 - 9 - 2) / 17 \rfloor = 4$ devices. Raising `MAX_DEVICES` past 4 causes a silent buffer overrun — confirmed in Issue #81. No bounds checking.
- **Broadcast `POLL` collisions.** All anchors attempt to reply to `POLL_ACK` simultaneously; only one wins per exchange, starving others.
- **Volatile bug.** `_networkDevicesNumber` not declared `volatile`; compiler may cache stale value, corrupting the device list (Issue #179, unfixed).
- **NLOS data unused.** `getFirstPathPower()` and `getReceivePower()` are implemented and stored on each device object, but **the difference is never computed, no NLOS score is derived, and no weighting or rejection logic exists.**

2.2.2 Makerfabs mf_DW1000

Thotro fork with ESP32 pin remapping. Law-of-cosines solver hardcoded for exactly 3 anchors. All thotro limitations apply unchanged.

2.2.3 jremington/UWB-Indoor-Localization_Arduino

The repo bundles two distinct library copies:

DW1000_library (thotro derivative): Adds volatile fix to `_networkDevicesNumber`; raises `MAX_DEVICES` to 7. However, the RANGE frame encoding is **identical** — still 17 bytes/device, $\lfloor 79/17 \rfloor = 4$ fit before overflow. Setting `MAX_DEVICES = 7` without changing the frame is a latent buffer overrun: devices 5–7 silently corrupt memory.

DW1000_library_pizzo00 (Pizzolato fork, hosted here since pizzo00’s original repo is deleted): Reduces per-device bytes to 12 using **differential** timestamps (`rangeDeviceSize = 12`), capping at `devicePerTransmit = 6` per frame ($\lfloor 79/12 \rfloor = 6$). This fixes the frame-size arithmetic for DW1000. Still uses broadcast POLL (RF collisions remain). Includes on-device LLS solver with precomputed $(A^T A)^{-1}$ (good technique, borrowed in our Phase 6.3).

2.2.4 F-Army/arduino-dw1000-ng (archived November 2023)

Architecture is fundamentally different: individual unicast TWR per anchor (no shared broadcast frame). The `tagRangeInfrastructure()` function sequences anchors one at a time via a `next_anchor` field in each anchor’s confirm reply. Consequently, there is no frame-size anchor ceiling. However: single-tag only, no NLOS detection, archived and unmaintained.

Corrected novelty claim on anchor scaling. The thotro/Makerfabs/jremington lineage has a **hard 4-anchor overflow boundary** in the shared RANGE frame. The pizzo00 fork fixes the frame math to reach 6. The F-Army dw1000-ng library also has no anchor ceiling via individual unicast TWR, but is archived (2023), single-tag only, and has no NLOS detection or IMU fusion.

Our system is the first to combine: (1) individual addressed DS-TWR with no anchor ceiling, (2) multi-tag TDMA design, (3) FP_POWER NLOS detection, and (4) IMU fusion — in a single open-source library.

2.2.5 NLOS Detection — Verified Novelty

Confirmed: no existing open-source DW1000 positioning library uses FP_POWER for NLOS detection.

Every audited library (thotro, Makerfabs, jremington/DW1000_library, jremington/DW1000_library_pizzo00, F-Army/dw1000-ng, linorobot/ros_dwm1000) calls `getFirstPathPower()` and `getReceivePower()` and stores both values on the device object — but **none computes their difference, none derives an NLOS score, and none uses that score to weight or reject a measurement.**

The F-Army library has an `_nlos` boolean flag, but it controls the LDE leading-edge algorithm configuration (a static chip setting), not per-packet NLOS inference.

Academic papers (e.g. arXiv:2403.19706) use the APS006 FP_power metric in post-processing Python/MATLAB scripts, not in the embedded Arduino positioning library.

Our implementation of per-packet $\Delta P_{\text{NLOS}} = P_{\text{RX}} - P_{\text{FP}}$ with weighted multilateration is, to our knowledge, the first in an open-source DW1000 positioning library.

2.3 Techniques Borrowed from Existing Libraries

Table 2: Techniques incorporated from existing work

Source	Technique	Our phase
jremington	Age-gated anchor eviction (MATLAB)	Phase 0 (✓ Done)
jremington / pizzo00	On-device LLS with precomputed $(A^T A)^{-1}$	Phase 6.3
pizzo00	Differential timestamp encoding (12 B vs 17 B/device)	Phase 6.2
dw1000-ng	Deep sleep on anchors between ranging cycles	Phase 5.1
Nobody	FP_POWER / RX_POWER NLOS score & weighting	Phase 2.1 (novel)
Nobody	Anchor self-survey via MDS	Phase 1.5 (novel)
Nobody	Full 6-DOF (XYZ + RPY) with UWB + IMU fusion	Phase 4 (novel)

3 Implementation Status

Phase 0 — Already Complete

- ✓ MATLAB watchdog + UDP auto-reconnect (10 s timeout → recreates `udpport`)
- ✓ EKF divergence guard + auto-reset (jump >2 m or $\text{tr}(P) > 25$ triggers reinit)
- ✓ Age-gated anchor eviction in MATLAB (`AGE_GATE_SEC = 3.0 s`)
- ✓ `RANGE_BIAS_M` tuning slider (0–0.10 m global multipath bias subtraction)
- ✓ Inner try-catch (bad sweeps skip silently, session survives)
- ✓ Status label in tuning panel (packet count, EKF resets, current bias)
- ✓ Live 5-slider tuning panel with Stationary / Slow Walk / Fast Walk presets

4 Phase 1 — Foundation Fixes

Phase 1 — Foundation | No algorithm changes. Highest ROI for effort.

Attacking calibration, protocol, and configuration issues before adding complexity. Expected improvement: 3–10 cm → 2–6 cm XY.

4.1 1.0 Switch to 64 MHz PRF Accuracy Mode

Easy

Change: `DW1000.enableMode(MODE_LONGDATA_RANGE_LOWPOWER) → MODE_LONGDATA_RANGE_ACCURACY` in `UwbConfig.h`.

Why: 64 MHz PRF produces a sharper autocorrelation peak in the channel impulse response. The receiver can more reliably identify the first-path arrival and reject multipath peaks arriving slightly later. The Pro boards already have a PA/LNA amplifier for range — this switch improves accuracy, not range.

Actions: Reflash all 4 boards. Reduce reply delay `UWB_REPLY_DELAY_US 7000 → 6000` μs . Re-run calibration (1.1) — delay register values differ between PRF modes.

Removes: 1–3 cm systematic ranging error from multipath peak ambiguity.

4.2 1.1 Better Calibration

Easy

Change: `SAMPLES_PER_STEP 40 → 200`; `SEARCH_ITERS 12 → 14` in `AntennaCalibration.ino`.

Why: 40 samples gives ± 1 cm statistical uncertainty on the mean, propagating to ± 3 delay-register units of systematic error. 200 samples reduces this to ± 0.3 cm.

Validation: After convergence run the calibration loop for 60 s and record mean $\pm$ std. Validate at two distances (e.g. 1.5 m and 5 m) — both should agree within ± 2 cm.

Removes: ± 1 – 3 cm systematic calibration bias per board.

4.3 1.2 Adaptive Polling + Age-Gated Anchor Eviction

Easy

Firmware (remaining): Add `failCount[N]` per anchor in `UwbScheduler`. After 3 consecutive timeouts, skip that anchor for 5 sweeps then retry with exponential backoff. This avoids wasting 100 ms per dead anchor per sweep.

MATLAB: ✓ **Done** — `anchorLastSeen` struct with `tic/toc` tracking; `AGE_GATE_SEC = 3.0` evicts stale range history automatically.

4.4 1.3 Faster Sweep Rate

Medium

Option A (safe): Reduce `UWB_REPLY_DELAY_US` 7000 $\rightarrow$ 5500 μ s. Verify exchanges still complete.

Option B (aggressive): Switch to `MODE_SHORTDATA_FAST_LOWPOWER` (6.8 Mbps, 64-symbol preamble). Reply delay drops to $\sim 1500 \mu$ s giving 8–10 Hz for 3 anchors. Cost: ~ 2 m range reduction (acceptable for ≤ 15 m indoor use).

Higher sweep rate directly improves all filter layers downstream.

4.5 1.4 Multipath Bias Correction

✓ **Done**

Global `RANGE_BIAS_M` subtracted from all filtered ranges before the multilateration. Tunable 0–0.10 m via slider. Tune by placing tag at a known point and minimising `info.rms`.

4.6 1.5 Anchor Self-Survey (Auto-Localization)

Medium

What: Anchors range to each other. MATLAB computes the anchor layout via Multidimensional Scaling (MDS) and writes `anchors.json` automatically. No tape measure required.

Firmware: New sketch `examples/AnchorSurvey/AnchorSurvey.ino`. Each anchor temporarily takes the tag role and ranges to all other anchor addresses. Sends `SURVEY,v1,<src>,<dst>,<dist_mm>,<quality>` over HostLink.

MATLAB: New script `matlab/runSurvey.m`. Collects $\binom{N}{2}$ pairwise distances (averaged over 100 sweeps per pair). Runs classical MDS: build squared-distance matrix $D^{(2)}$, apply double-centering $B = -\frac{1}{2}JD^{(2)}J$, eigendecompose, take top- d eigenpairs. Fix frame (anchor 1 at origin, anchor 2 on $+X$, anchor 3 in $+Y$ half-plane). Write `anchors.json`.

Accuracy: ≈ 1 – 2 cm anchor position accuracy after 100-sample averaging ($\sigma_d/\sqrt{100} \approx 0.5$ cm per pair).

Limitation: 3D self-survey requires meaningful height variation. All-same-height gives a degenerate Z axis.

5 Phase 2 — Ranging Quality

Phase 2 — Ranging Quality | Firmware + MATLAB. Attacks measurement errors at source.

Expected improvement: 2–6 cm → 1–4 cm XY.

5.1 2.1 FP_POWER NLOS Detection**Medium ★ Novel**

What: Read first-path amplitude registers after each DW1000 receive. Compute per-packet NLOS score per Decawave APS006. Send in RANGE_REPORT payload. Parse and act in MATLAB.

Registers: FP_AMPL1, FP_AMPL2, FP_AMPL3 (reg 0x12, offsets 0x07–0x0C); RXPACC (reg 0x10, offset 0x2C); CIR_PWR (reg 0x12, offset 0x00).

Math:

$$P_{FP} = 10 \log_{10} \left(\frac{A_1^2 + A_2^2 + A_3^2}{N_{acc}^2} \right) - A_{const} \quad (A_{const} = 113.77 \text{ dBm at } 64 \text{ MHz PRF})$$

$$\Delta P_{NLOS} = P_{RX} - P_{FP}$$

Score	Condition	Action
< 3 dB	LOS	Full weight
3–6 dB	Soft NLOS	Reduce weight
> 6 dB	Hard NLOS	Strong downweight or reject

Why it matters: Detects NLOS before the solver runs, based on physics. Works even when consistent NLOS bias looks “normal” to MAD gating.

Novelty: Confirmed by auditing all major DW1000 libraries (thotro, Makerfabs, jremington, pizzo00, dw1000-ng, linorobot). Every library exposes `getFirstPathPower()` and `getReceivePower()` but **none computes their difference, derives an NLOS score, or uses it to weight or reject measurements.**

5.2 2.2 Weighted Multilateration**Medium**

Derive weights from NLOS score:

$$\sigma_i = \sigma_{base} \cdot 10^{\Delta P_{NLOS,i}/20}, \quad w_i = 1/\sigma_i^2$$

A 10 dB NLOS anchor receives 1/100 the weight of a clean LOS anchor. Modify Levenberg-Marquardt solver in `Multilateration.m` to use weighted residuals and Jacobian $\mathbf{J}^T \mathbf{W} \mathbf{J}$.

5.3 2.3 NLOS Range Bias Correction**Easy**

Physical bias correction applied per anchor before the solver:

$$d_{corrected} = d_{measured} - k_{NLOS} \cdot \max(0, \Delta P_{NLOS} - 3 \text{ dB})$$

Tunable slider K_{NLOS} (0–0.06 m/dB) in the tuning panel. Removes systematic positive through-wall ranging bias (typically 20–50 cm per obstructed wall).

Requires: Phase 2.1 for the per-packet NLOS score.

5.4 2.4 Per-Anchor Diagnostics Panel**Easy**

Second axes in `LivePlotter` showing per-anchor range residual (bar chart) and NLOS score (colour-coded: green/yellow/red) updated every sweep. Provides immediate visual feedback on which anchor is causing problems.

5.5 2.5 GPS-Style δ -Solve (4+ Anchors)

Medium

Augment the multilateration state to $[\mathbf{x}^\top, \delta]^\top$ where δ is a common range bias:

$$\hat{d}_i = \|\mathbf{x} - \mathbf{a}_i\| + \delta + e_i$$

Jacobian row i : $[\text{unit_vec}_i, 1]$. Solved δ is a real-time calibration health metric.

Warning: With $N = d + 1$ anchors (3 in 2D) this reduces to TDOA geometry — ill-conditioned near the anchor centroid. Only enable with $N \geq d + 2$ anchors.

Requires: Phase 4.1 (4th anchor).

6 Phase 3 — State Estimation

Phase 3 — State Estimation | Better filter models. MATLAB only.

Expected improvement: 1–4 cm $\rightarrow$ 1–3 cm XY.

6.1 3.1 Constant-Acceleration EKF (CV $\rightarrow$ CA Model)

Medium

Upgrade from 4-state $[\mathbf{p}, \mathbf{v}]$ to 6-state $[\mathbf{p}, \mathbf{v}, \mathbf{a}]$. Dynamics: position $\leftarrow$ position + velocity $\cdot \Delta t + \frac{1}{2}$ acceleration $\cdot \Delta t^2$; acceleration $\leftarrow$ acceleration + random jerk (Singer model). Better tracks direction changes and acceleration phases; reduces lag on turns.

6.2 3.2 δ as EKF State (Works with 3 Anchors)

Med-Hard

Add δ to EKF state: $[\mathbf{p}, \mathbf{v}, \delta]$ or $[\mathbf{p}, \mathbf{v}, \mathbf{a}, \delta]$. δ evolves as a random walk. Tight coupling: raw ranges feed directly into EKF measurement update. Measurement: $h_i(\mathbf{x}) = \|\mathbf{p} - \mathbf{a}_i\| + \delta$. Jacobian row i : $[\hat{u}_i, \mathbf{0}, 1]$ (unit vector + zeros for velocity/accel + 1 for bias).

Advantage over multilaterator δ -solve: Works with 3 anchors; temporally smoothed; avoids ill-conditioning near centroid.

6.3 3.3 Moving Horizon Estimator (MHE)

Hard

Constrained optimisation over a sliding window of N past sweeps. At time k , minimise:

$$\min_{\{\mathbf{x}_t\}} \underbrace{\|\mathbf{x}_{k-N+1} - \bar{\mathbf{x}}_{k-N+1}\|_{\mathbf{P}_{\text{arr}}^{-1}}^2}_{\text{arrival cost}} + \underbrace{\sum_t \|\mathbf{x}_t - \mathbf{F}\mathbf{x}_{t-1}\|_{\mathbf{Q}^{-1}}^2}_{\text{dynamics}} + \underbrace{\sum_t \sum_i \rho_\delta(\|\mathbf{p}_t - \mathbf{a}_i\| - \hat{d}_{t,i})}_{\text{range residuals (Huber)}}$$

subject to $\mathbf{p}_{\min} \leq \mathbf{p}_t \leq \mathbf{p}_{\max}$ (room boundary) and $\|\mathbf{v}_t\| \leq v_{\max}$ (speed limit).

Huber loss threshold $\delta = 0.15$ m. Computation: `fmincon` with $N = 6$, $d = 4$ states $\Rightarrow$ 2–5 ms per solve.

Advantages over EKF: NLOS absorbed by Huber loss; room boundary prevents wild jumps; physically impossible positions excluded.

6.4 3.4 Hybrid EKF + MHE

Medium

EKF every sweep (real-time, zero lag). MHE every $K = 5$ sweeps (accuracy pass). If outputs diverge beyond threshold, flag increased uncertainty. Requires Phase 3.3 complete.

7 Phase 4 — 3D + Orientation (XYZ + RPY)

Phase 4 — 3D + RPY | Firmware + MATLAB. Adds the 3rd dimension and full orientation.

Expected outcome: XY 1–3 cm, Z 2–5 cm, RPY 3–5° at 100 Hz.

7.1 4.1 4th Anchor + 3D Configuration

Easy

Flash board 4 as `ANCHOR_ID = 0x04`. Mount at a different height from the others. Update `anchors.json`: add entry with non-zero z , set `"dim": 3`. Add `0x04` to `ANCHORS[]` in `Tag.ino`. This also enables the δ -solve (Phase 2.5) since we now have $d + 2 = 5$ constraints.

7.2 4.2 BNO085 IMU — Firmware Implementation

Medium

Implement `SensorImu::begin()` and `SensorImu::read()` using the Adafruit BNO08x SPI library.

Why SPI: The I2C bus (pins 4/5) is occupied by the SSD1306 OLED. The BNO085 also has known clock-stretching issues on ESP32 I2C.

Reports: ARVR Stabilised Rotation Vector at 10 ms intervals (quaternion, 100 Hz) + Linear Acceleration at 10 ms (gravity removed, 100 Hz). The IMU tail in `HostLink` and the parser in `FrameParser.m` are already wired for this data.

7.3 4.3 RPY Output from Quaternion

Easy

Extract Euler angles from the BNO085 quaternion $[q_w, q_x, q_y, q_z]$ using the ZYX convention:

$$\begin{aligned}\phi &= \text{atan2}\left(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)\right) && \text{(roll)} \\ \theta &= \arcsin(2(q_w q_y - q_z q_x)) && \text{(pitch)} \\ \psi &= \text{atan2}\left(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)\right) && \text{(yaw)}\end{aligned}$$

Log RPY alongside XYZ. Add RPY readout to HUD. The magnetometer in the BNO085 provides an absolute yaw reference, preventing unbounded yaw drift.

Requires: Phase 4.2.

7.4 4.4 Loose Coupling — IMU-Aided EKF

Hard

IMU predict step at 100 Hz (fills the 333 ms gaps between UWB sweeps). UWB measurement update at 3 Hz corrects accumulated drift.

State: $[\mathbf{p}, \mathbf{v}, q_w, q_x, q_y, q_z, \delta]$ (11-state, 3D). IMU prediction:

$$\begin{aligned}\mathbf{a}^w &= \mathbf{R}(\mathbf{q}) \mathbf{a}^b \quad (\text{rotate to world frame}) \\ \mathbf{p}_{k+1} &= \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k^w \Delta t^2 \\ \mathbf{q}_{k+1} &= \mathbf{q}_k \otimes \exp\left(\frac{\Delta t}{2} \boldsymbol{\omega}_k^b\right)\end{aligned}$$

Effectively decouples position accuracy (UWB) from position smoothness (IMU).

7.5 4.5 Tight Coupling — Raw Ranges + IMU in One Filter Very Hard

State: $[\mathbf{p}, \mathbf{v}, \mathbf{q}, \delta]$ (11-state). IMU drives prediction at 100 Hz. Raw UWB ranges (not multi-laterated position) are the measurement at 3 Hz. Solver: MHE with Huber loss, room bounds, speed limit, and quaternion unit-norm constraint $\|\mathbf{q}\| = 1$.

This is the architecture commercial UWB positioning systems use on dedicated hardware. Full 6-DOF output: XYZ + RPY + velocity + bias estimate.

8 Phase 5 — System Hardening

8.1 5.1 Deep Sleep on Anchors (from dw1000-ng) Medium

Put DW1000 into deep sleep for $\sim 80\%$ of the sweep period between ranging cycles. Reduces anchor current from ~ 100 mA continuous to ~ 20 mA average — significant for battery-powered deployment.

8.2 5.2 DW1000 Driver Error Handling Medium

The vendored driver contains 40+ `// TODO proper error/warning handling` comments. Add recovery for SPI lockup, PLL failure, and LDE timeout. Auto-reset radio if it goes silent for > 500 ms.

8.3 5.3 Temperature Compensation Medium

The DW1000 on-chip temperature sensor is OTP-calibrated and already read by the driver but never applied. Clock drift is approximately 0.1 ticks/ $^{\circ}\text{C} \approx 0.5$ mm/ $^{\circ}\text{C}$ per board. Apply a linear correction relative to the calibration temperature.

8.4 5.4 Online Self-Calibration Hard

Batch optimisation jointly solving for anchor positions and per-board delays using accumulated tag trajectory + range data. Requires spatial diversity (tag must visit different areas of the room). Replaces the manual calibration procedure entirely.

9 Phase 6 — Scale and Future

9.1 6.1 Multi-Tag TDMA Superframe Hard

Time-slotted TDMA. Master anchor broadcasts slot assignments via ANNOUNCE frames (type byte already defined in `UwbFrame.h`). MATLAB maintains a separate EKF/MHE per `tagId`. Requires careful timing budget: $N_{\text{anchors}} \times N_{\text{tags}}$ TWR exchanges per superframe.

9.2 6.2 Differential Timestamp Encoding (pizzo00 technique) Hard

If a broadcast POLL mode is ever needed (e.g. for firmware-side solving), adopt theizzo00 12-byte/device encoding (two differential timestamps instead of three absolute) to reach 6 anchors per RANGE frame.

9.3 6.3 On-Device 2D Solve (jremington technique) Medium

Precompute $(A^{\top}A)^{-1}$ once at startup (anchors are fixed). Position update becomes a matrix-vector multiply. Display (x, y) on the OLED without a host PC. Useful for standalone deployment.

10 Full Item Index

Item	Description	Difficulty	Status
<i>Phase 0 — Already Done</i>			
0.1	MATLAB watchdog + UDP auto-reconnect	—	✓ Done
0.2	EKF divergence guard + auto-reset	—	✓ Done
0.3	Age-gated anchor eviction (MATLAB)	—	✓ Done
0.4	RANGE_BIAS_M tuning slider	—	✓ Done
0.5	Inner try-catch (bad sweeps skip, session survives)	—	✓ Done
0.6	Status label in tuning panel	—	✓ Done
<i>Phase 1 — Foundation</i>			
1.0	Switch to 64 MHz PRF (MODE_LONGDATA_RANGE_ACCURACY)	Easy	Pending
1.1	Better calibration (200 samples, multi-distance)	Easy	Pending
1.2	Adaptive polling — skip dead anchors (firmware)	Easy	Partial
1.3	Faster sweep rate (reduce reply delay / mode change)	Medium	Pending
1.5	Anchor self-survey + auto <code>anchors.json</code>	Medium	Pending
<i>Phase 2 — Ranging Quality</i>			
2.1	FP_POWER NLOS detection (firmware + wire format)	Medium	Pending
2.2	Weighted multilateration (NLOS score → weights)	Medium	Pending
2.3	NLOS range bias correction slider	Easy	Pending
2.4	Per-anchor diagnostics panel in LivePlotter	Easy	Pending
2.5	GPS-style δ -solve in Multilaterator (4+ anchors)	Medium	Pending
<i>Phase 3 — State Estimation</i>			
3.1	Constant-acceleration EKF (CV → CA model)	Medium	Pending
3.2	δ as EKF state (bias estimation, 3 anchors OK)	Med-Hard	Pending
3.3	MHE (sliding window, Huber loss, room constraints)	Hard	Pending
3.4	Hybrid EKF + MHE	Medium	Pending
<i>Phase 4 — 3D + RPY</i>			
4.1	4th anchor + 3D <code>anchors.json</code>	Easy	Pending
4.2	BNO085 <code>SensorImu</code> implementation (SPI)	Medium	Pending
4.3	RPY output from quaternion	Easy	Pending
4.4	Loose coupling — IMU-aided EKF (100 Hz predict)	Hard	Pending
4.5	Tight coupling — raw ranges + IMU in MHE	Very Hard	Pending
<i>Phase 5 — System Hardening</i>			
5.1	Deep sleep on anchors	Medium	Pending
5.2	DW1000 driver error handling	Medium	Pending
5.3	Temperature compensation	Medium	Pending
5.4	Online self-calibration	Hard	Pending
<i>Phase 6 — Scale</i>			
6.1	Multi-tag TDMA superframe	Hard	Pending
6.2	Differential timestamp encoding	Hard	Pending
6.3	On-device 2D solve (OLED display)	Medium	Pending

11 Expected Accuracy at Each Phase

Table 4: Expected accuracy progression through implementation phases

After	XY	Z	RPY	Rate	Key enabler
Current	3–10 cm	—	—	3 Hz	Calibration noise + NLOS + CV EKF
Phase 1	2–6 cm	—	—	4–8 Hz	Calibration bias removed, PRF improved
Phase 2	1–4 cm	—	—	4–8 Hz	NLOS weighted/rejected before solver
Phase 3	1–3 cm	—	—	4–8 Hz	Better filter model, MHE room constraints
Phase 4	1–3 cm	2–5 cm	3–5°	100 Hz RPY	IMU fills gaps, 3D anchor geometry
Phase 5+	1–3 cm	2–5 cm	3–5°	100 Hz RPY	Reliability, power, robustness

12 Recommended Implementation Sequence

1.0	→	1.1	→	1.2	→	1.5	→	1.3	→	2.1	→	2.2
PRF		Cal		Poll		Survey		Rate		NLOS		Wgt
→	2.4	→	3.1	→	4.1	→	4.2	→	4.3	→	4.4	
	Plot		CA-EKF		4th Anchor		IMU		RPY		Fusion	

Phases 2.5, 3.2, 3.3, 3.4, 4.5, and all of Phases 5–6 can be added in any order after 4.4 is stable.